

Tetris AI: Specific Algorithms & Methodologies

When documenting or discussing a Tetris AI, it is crucial to distinguish between broad algorithmic paradigms (e.g., "Machine Learning," "Search") and the specific, named algorithms implemented within the code.

Based on the architecture of your Tetris AI Simulator, here are the actual, specific algorithms related to each aspect of the system.

1. Move Selection & Decision Making

The aspect of the code that simulates drops and chooses the highest-scoring column and rotation.

Broad Category: State Space Search / Action Selection **Specific Algorithms:**

- **Greedy Best-First Search:**
 - **How it applies:** Your current AI looks exactly one move ahead (1-ply). It evaluates all immediate children (every possible rotation and column drop) and strictly chooses the state with the highest immediate heuristic score. It does not look further down the tree.
- **Expectimax Algorithm:**
 - **How it applies:** If your AI were to look ahead at the *next* piece (which is randomly generated), standard Minimax wouldn't work because there is no adversary. Instead, you use Expectimax. The "Max" node tries to maximize the score of the current piece, and the "Chance" node calculates the expected value of the random next piece coming from the bag.
- *A (A-Star) Pathfinding Algorithm:**
 - **How it applies:** Currently, your AI assumes a piece can magically drop straight down into its target column (`checkCollision` on a direct vertical path). In a real Tetris game, you must navigate pieces under overhangs (tucks and spins). A* is the specific algorithm used to pathfind the tetromino through the grid from the spawn point to the target position, treating the grid as a graph.

2. Board Evaluation

The aspect of the code that uses `linesWeight`, `heightWeight`, `holesWeight`, and `bumpinessWeight` to assign a score to a grid.

Broad Category: Heuristic Evaluation Functions **Specific Algorithms/Models:**

- **Pierre Dellacherie's Algorithm (The Dellacherie Heuristic):**

- **How it applies:** In Tetris AI research, feature-based evaluation is often tied to specific researchers who identified the most mathematically sound features. Your AI uses a simplified subset of the Dellacherie feature set. Dellacherie's specific algorithm evaluates 6 specific metrics: Landing Height, Eroded Piece Cells, Row Transitions, Column Transitions, Number of Holes, and Board Wells. Naming this shows you understand the historical literature of Tetris AI.
- **Thiery & Scherrer's BZ (Building & Zapping) Strategy:**
 - **How it applies:** This is a specific algorithmic approach to board evaluation that separates the game into two phases: "Building" (optimizing for structure and minimizing holes without clearing lines) and "Zapping" (clearing lines to survive).

3. Weight Optimization & Training

The aspect of figuring out that the optimal weights are 0.76 , -0.51 , -0.35 , -0.18 .

Broad Category: Machine Learning / Evolutionary Computation **Specific Algorithms:**

- **Cross-Entropy Method (CEM):**
 - **How it applies:** If you were asked, "How did you find these weights?", CEM is one of the most famous algorithms used for Tetris. It is a specific Monte Carlo algorithm used for rare-event simulation and multi-extremal optimization. It works by generating a population of weight vectors, playing games, selecting the top performers, and updating a probability distribution to generate the next batch.
- **CMA-ES (Covariance Matrix Adaptation Evolution Strategy):**
 - **How it applies:** This is a specific, highly efficient numerical optimization algorithm. If you were using a "Genetic Algorithm," CMA-ES is the actual state-of-the-art algorithm you would implement to mutate and recombine the weights of your heuristics over thousands of generations.
- **TD(λ) (Temporal Difference Learning):**
 - **How it applies:** If your AI was learning the weights dynamically by playing the game rather than using a static evolutionary approach, TD(λ) is the specific Reinforcement Learning algorithm you would use. It assigns credit/blame to past board states based on a delayed reward (e.g., getting a Game Over or clearing 4 lines).

Summary for your Consultation

If asked to describe the AI, you can now say:

*"The core engine utilizes a **Greedy Best-First Search** to navigate the state space of immediate piece placements. To evaluate these states, we use a feature-based linear combination inspired by **Dellacherie's Algorithm**, focusing on aggregate height, hole count, and bumpiness. In the future, these heuristic*

*weights can be optimized using the **Cross-Entropy Method (CEM)** or **CMA-ES** rather than manual tuning."*